

12주차 07 트랜스포머

트랜스포머란?

트랜스포머(Transformer)는 2017년 코넬 대학의 아시시 바스와니(Ashish Vaswani) 등의 연구 그룹이 발표한 「Attention is All You Need」 논문을 통해 소개된 신경망 아키텍처다.

트랜스포머 모델의 주요 기능 중 하나는 기존의 순환 신경망과 같은 순차적 방식이 아닌 병렬로 입력 시퀀스를 처리하는 기능이다. 긴 시퀀스의 경우 트랜스포머 모델을 순환 신경망 모델보다 훨씬 더 빠르고 효율적으로 처리한다.

트랜스포머 모델의 구조가 기존의 **순차 처리(Sequential Processing)**나 **반복 연결(Recurrent Connections)**에 의존하지 않고 입력 토큰 간의 관계를 직접 처리하고 이해할 수 있도록 하는 **셀프 어텐션(Self-Attention)**을 기반으로 하기 때문이다. 이로 인해 모델이 재귀나 합성곱 연산 없이 입력 토큰 간의 관계를 직접 모델링할 수 있다.

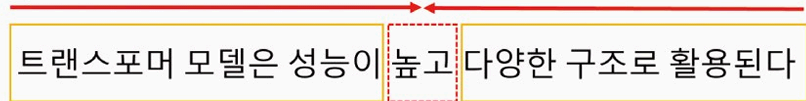
트랜스포머 모델의 학습은 대용량 데이터셋에서 매우 효율적이며 데이터의 양이 많은 기계 번역과 같은 작업에 적합하다. 언어 모델링 및 텍스트 분류와 같은 작업에서 매우 효과적인 것으로 나타났으며, 광범위한 자연어 처리 작업에서 높은 효율을 보인다. 특히 기계 번역, 언어 모델링, 텍스트 요약과 같은 장기적인 종속성을 포함하는 작업에 주로 사용됐으며, 자연어 처리 분야에서 널리 사용되고 영향력이 큰 모델이 됐다.

학습 구조

이번 장에서 소개하는 트랜스포머 기반 모델들은 **오토 인코딩(Auto-Encoding)**, **자기 회귀(Auto Regressive)** 방식 또는 두 개의 조합으로 학습된다.

오토 인코딩 방식은 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업(Task)을 수행한다. 예측되는 토큰의 양옆에 있는 토큰들을 참조하기 때문에 양방향 구조를 가지며 이를 **인코더(Encoder)**라고 한다.

A) 양방향 구조



B) 단방향 구조

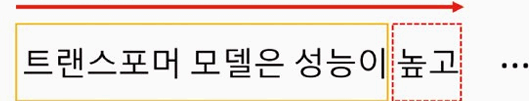


그림 7.1 트랜스포머 구조

그림 7.1의 A는 양방향 구조를 보여준다. 반면에 자기 회귀 방식은 이전 단어들이 주어졌을 때 다음 단어가 무엇인지 맞히는 작업을 수행한다.

그림 7.1의 B는 단방향 구조를 보여준다. 예측되는 토큰의 왼쪽에 있는 토큰들만 참조하기 때문에 단방향 구조를 가지며 이를 **디코더(Decoder)**라고 한다.

이번 장에서 소개하는 트랜스포머 모델들은 양방향성 구조의 인코더(오토 인코딩) 또는 단방향성 디코더(자기 회귀)를 사용하는 공통적 특징을 가지고 있다.

트랜스포머 모델 구조

모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

모델 특징을 정리하면 다음의 표 7.1과 같다. ELECTRA는 인코더에 생성적 적대 신경망의 판별기를 활용한 모델이며, T5와 BART는 다양한 자연어 처리 작업에 대해 학습을 수행한다.

Transformer

트랜스포머(Transformer)는 딥러닝 모델 중 하나로, 기계 번역, 챗봇, 음성 인식 등 다양한 자연어 처리 분야에서 많은 성과를 내는 모델이다. 이 모델은 순환 신경망이나 합성곱 신경

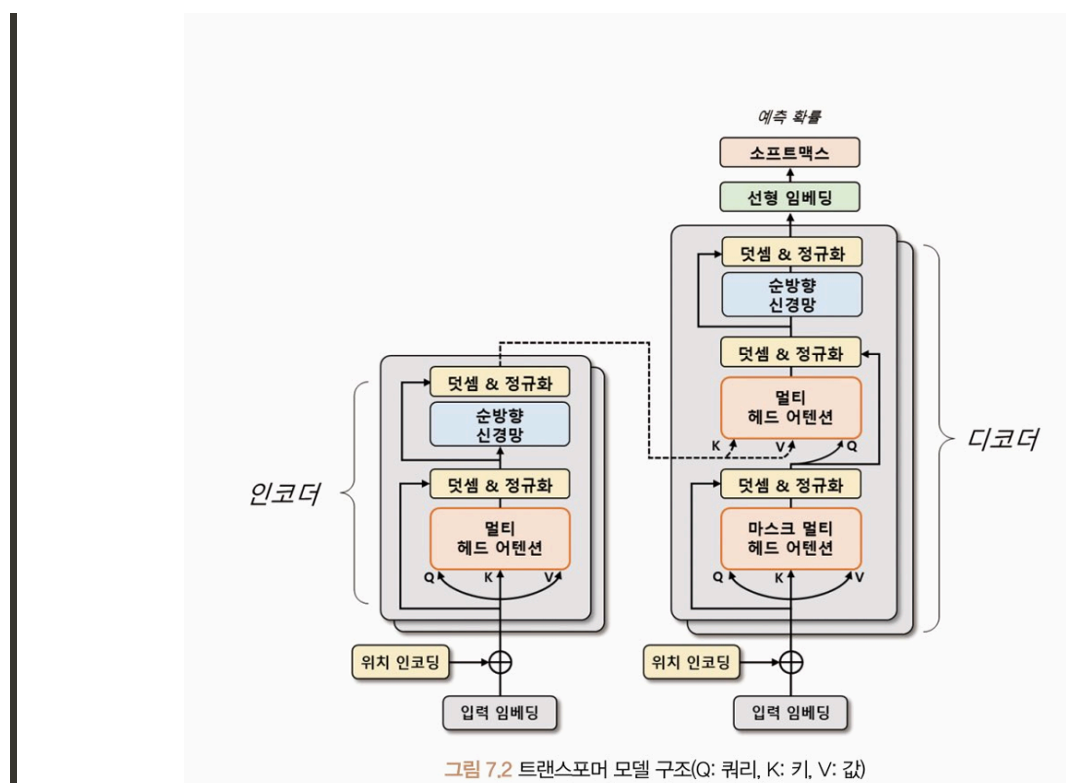
망과 달리 **어텐션 메커니즘(Attention Mechanism)**만을 사용하여 시퀀스 임베딩을 표현한다.

트랜스포머의 어텐션 메커니즘은 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞추어 문맥을 이해하고 적절한 출력을 생성한다. 인코더는 입력 시퀀스를 임베딩하여 고차원 벡터로 변환하고, 디코더는 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성한다. 이때 어텐션 메커니즘은 인코더와 디코더 단어 사이의 상관관계를 계산하여 중요한 정보에 집중한다. 이를 통해 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련이 있는지를 파악하여 번역이나 요약문 생성과 관련된 작업 등을 수행할 수 있게 된다.

트랜스포머 모델은 기존의 순환 신경망 기반 모델보다 학습 속도가 빠르고, 병렬 처리가 가능해 대규모 데이터셋에서 높은 성능을 보인다. 또한, 임베딩 과정에서 문장의 전체 정보를 고려하기 때문에 문장의 길이가 길어지더라도 성능이 유지된다.

이러한 장점으로 인해 자연어 처리 분야에서 가장 인기 있는 모델이다.

트랜스포머 모델 구조



트랜스포머의 인코더와 디코더는 두 부분으로 구성돼 있으며, 각각 N개의 **트랜스포머 블록 (Transformer Block)**으로 구성된다. 이 블록은 **멀티 헤드 어텐션(Multi-Head Attention)**과 **순방향 신경망**으로 이뤄져 있다.

멀티 헤드 어텐션은 입력 시퀀스에서 쿼리(Query), 키(Key), 값(Value) 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션(Self-Attention)하는 벡터 표현 방법이다. 이 과정에서 쿼리와 각 키의 유사도를 계산하고, 해당 유사도를 가중치로 사용하여 값 벡터를 합산한다.

이렇게 계산된 어텐션 행렬은 입력 시퀀스 각 단어의 임베딩 벡터를 대체한다. 결국 입력 시퀀스의 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신한다.

순방향 신경망은 이 과정에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용된다. 이 신경망은 여러 개의 선형 계층으로 구성돼 있으며, 앞선 순방향 신경망의 구조와 동일하게 입력 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용한다. 이 과정에서 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신된다.

트랜스포머의 소스와 타겟 데이터

트랜스포머에서는 입력 시퀀스 데이터를 소스(Source)와 타겟(Target) 데이터로 나눠 처리한다. 예를 들어 영어를 한글로 번역하는 경우, 생성하는 언어인 한글을 타겟 데이터로 정의하고 참조하는 언어인 영어를 소스 데이터로 정의한다.

인코더는 소스 시퀀스 데이터를 위치 인코딩(Positional Encoding)된 입력 임베딩으로 표현해 트랜스포머 블록의 출력 벡터를 생성한다. 이 출력 벡터는 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성된다.

디코더도 인코더와 유사하게 트랜스포머 블록으로 구성되어 있지만, 마스크 멀티 헤드 어텐션(Masked Multi-Head Attention)을 사용해 타겟 시퀀스 데이터를 순차적으로 생성시킨다. 이때 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조한다.

최종적으로 생성된 디코더 출력 벡터는 선형 임베딩으로 재표현되어 이미지나 자연어 모델에 활용된다. 이러한 과정을 통해 트랜스포머는 챗봇, 기계 번역, 음성 인식 등 다양한 자연어 처리 작업에서 좋은 성능을 보인다.

입력 임베딩과 위치 인코딩

트랜스포머 모델에서 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환된다. 트랜스포머 모델은 순환 신경망과 달리 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보를 제공하지 않는다.

그러므로 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 한다. 이를 위해 트랜스포머는 위치 인코딩 방식을 사용한다.

위치 인코딩은 입력 시퀀스의 순서 정보를 모델에 전달하는 방법이다. 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보를 반영한다.

위치 인코딩 벡터는 $\sin$ 함수와 $\cos$ 함수를 사용해 생성되며, 이를 통해 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성한다. 위치 인코딩 벡터를 추가함으로써 모델은 단어의 순서 정보를 학습할 수 있게 된다.

위치 인코딩은 각 토큰의 위치를 각도로 표현해 $\sin$ 함수와 $\cos$ 함수로 위치 인코딩 벡터를 계산한다. 이러한 계산 방법은 토큰의 위치마다 동일한 임베딩 벡터를 사용하지 않기 때문에 각 토큰의 위치 정보를 모델이 학습할 수 있다.

위치 인코딩은 트랜스포머 모델에서 입력 시퀀스의 순서 정보를 보존하기 위한 중요한 방법 중 하나다.

수식 7.1 위치 인코딩

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

pos 는 입력 시퀀스에서 해당 단어의 위치를 나타내며, i 는 임베딩 벡터의 차원 인덱스를 의미한다. 차원의 인덱스가 짝수라면 첫 번째 $\sin$ 함수 수식을 적용하며, 홀수라면 두 번째 $\cos$ 함수 수식을 적용한다. 위치 인코딩에 사용되는 $1/10000^{(2i/d_{model})}$ 는 각도 정보로 변환하기 위한 스케일링 인자로 각 위치에 대한 주기적인 신호를 생성한다.

pe 변수의 텐서 차원은 $[50, 1, 128]$ 이 되며, $[최대 시퀀스, 1, 입력 임베딩의 차원]$ 을 의미한다. 출력 결과를 확인해 보면 위치별 임베딩 차원이 주기적인 값으로 구성되는 것을 확인할 수 있다. 산출된 위치 인코딩은 입력 임베딩과 더해진 후 드롭아웃이 적용된다.

특수 토큰

트랜스포머는 단어 토큰 이외의 특수 토큰을 활용하여 문장을 표현한다. 이 특수 토큰은 입력 시퀀스의 시작과 끝을 나타내거나 **마스킹(Masking)** 영역으로 사용된다.

특수 토큰으로 모델이 입력 시퀀스의 시작과 끝을 인식할 수 있게 하며, 마스킹을 통해 일부 입력을 무시할 수 있다. 예를 들어, 번역 모델에서는 디코더의 입력 시퀀스에서 현재 위치 이후의 토큰을 마스크해 이전 토큰만을 참조한다.

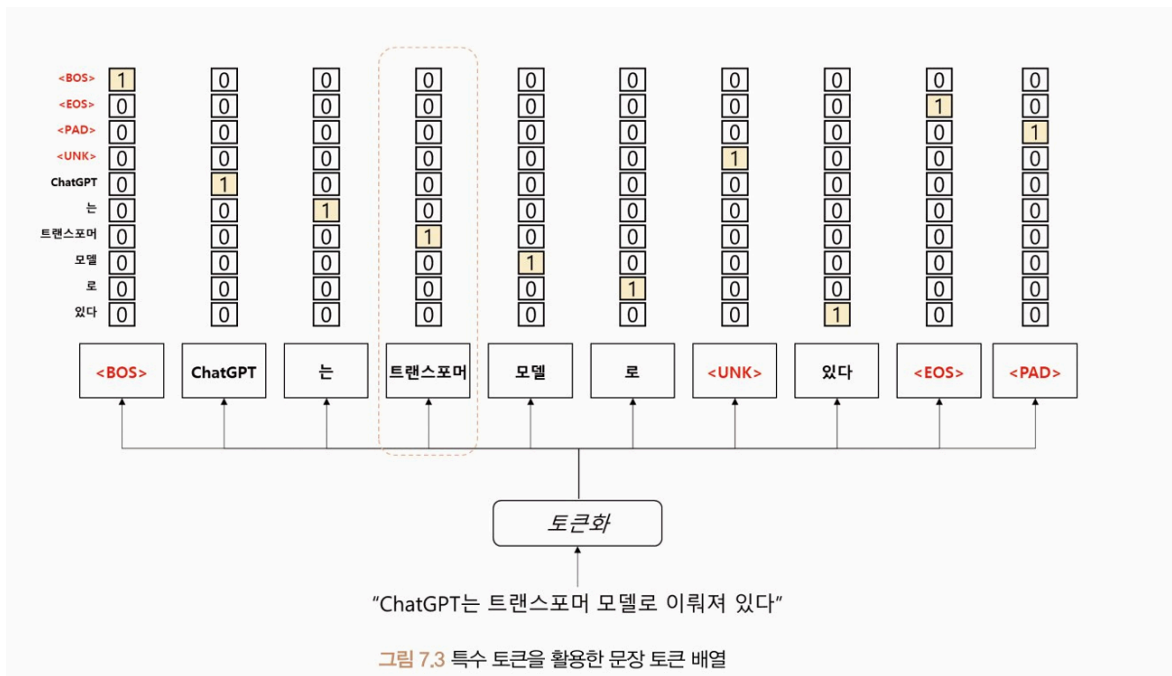
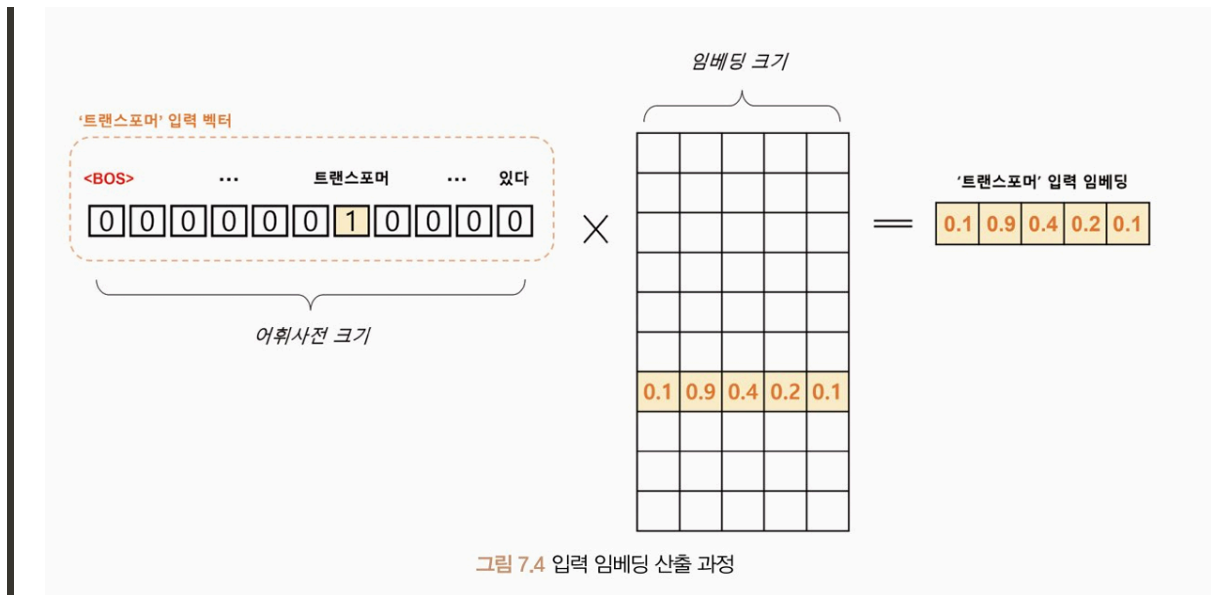


그림 7.3에서 사용한 **BOS(Beginning of Sentence)**, **EOS(End of Sentence)**, **UNK(Unknown)** 및 **PAD(Padding)** 토큰은 모두 특수 토큰으로 자연어 처리에서 일반적으로 사용된다.

- **BOS 토큰**: 문장의 시작을 나타내며, EOS 토큰은 문장의 종료를 나타낸다. 이 토큰들은 문장의 시작과 끝을 명확히 하기 위해 사용된다.
- **UNK 토큰**: 어휘 사전에 없는 단어로, 모르는 단어를 의미한다. 이 토큰은 모델이 이전에 본 적이 없는 단어를 처리할 때 사용된다.
- **PAD 토큰**: 모든 문장을 일정한 길이로 맞추기 위해 사용된다. 짧은 문장의 빈 공간을 채울 때 사용된다.

이렇게 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현한다.



입력 임베딩으로 변환되는 과정은 Word2Vec 방법과 동일하다. 어휘 사전의 크기를 V , 입력 임베딩 차원을 d 라고 했을 때, '트랜스포머'의 원-핫 벡터는 $[1, V]$ 크기를 갖는다. 이 원-핫 벡터는 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환된다.

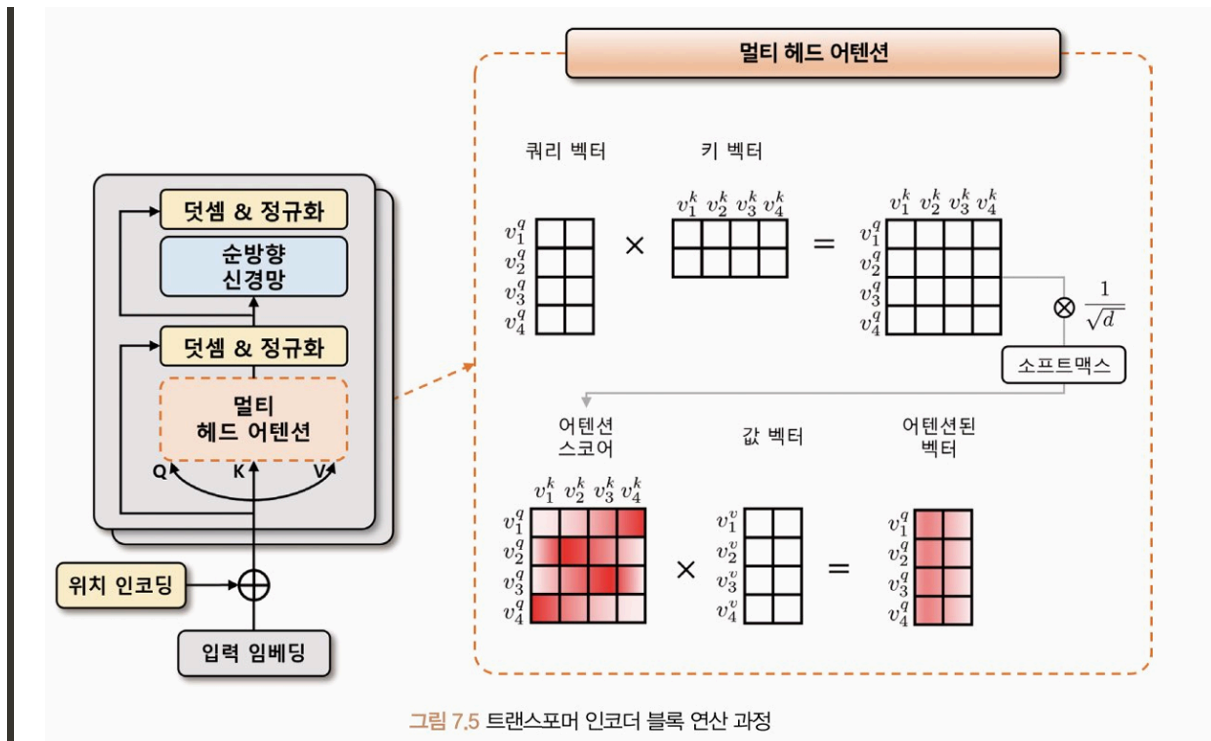
이러한 방식을 일반화하면, N 개의 문장이 최대 S 개의 토큰 길이를 가질 때 $[N, S, V]$ 크기의 원-핫 벡터 텐서는 $[N, S, d]$ 크기의 임베딩 텐서로 변환된다.

이 임베딩 텐서는 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유되는 텐서로 사용된다.

트랜스포머 인코더

트랜스포머 인코더는 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산을 수행한다. 각 인코더 계층은 멀티 헤드 어텐션과 순방향 신경망으로 구성되며, 입력 데이터에 대한 정보를 추출하고 다음 계층으로 전달한다.

인코더 계층에서 위치 정보를 반영하기 위해 위치 임베딩 벡터를 입력 벡터에 더해준다. 그리고 산출된 인코더 계층의 출력은 디코더 계층으로 전달된다.



트랜스포머 인코더는 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받는다. 멀티 헤드 어텐션 단계에서 입력 텐서 차원이 $[N, S, d]$ 라고 한다면 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터를 생성한다. 생성된 3개의 벡터는 각각 쿼리(Q), 키(K), 값(V) 벡터라고 정의한다.

- **쿼리 벡터(v^q):** 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터로, 인코더의 각 시점마다 생성된다. 현재 시점에서 질문이 되는 벡터를 의미하며 이 벡터를 기준으로 다른 시점의 정보를 참조한다.
- **키 벡터(v^k):** 쿼리 벡터와 비교되는 대상으로 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터가 된다. 키 벡터는 인코더의 각 시점에서 생성된다.
- **값 벡터(v^v):** 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할을 한다.

이러한 쿼리, 키, 값 벡터는 초기에 비슷한 값을 가지지만, 모델 학습을 통해 각 벡터는 의도한 의미의 값을 갖게 된다. 쿼리와 키 벡터의 연관성은 내적 연산으로 $[N, S, S]$ 어텐션 스코어 맵을 사용한다.

수식 7.2 어텐션 스코어 계산식

$$\text{score}(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

셀프 어텐션 과정에서는 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구하고 이 스코어값에 $\sqrt{d}$ (벡터 차원의 제곱근)만큼 나눠 보정한다. 이 보정값은 벡터 차원이 커질 때 스코어값이 같이 커지는 문제를 완화하기 위해 적용한다.

그리고 보정된 어텐션 스코어를 소프트맥스 함수를 이용하여 확률적으로 재표현하고, 이를 값 벡터와 내적하여 셀프 어텐션된 벡터를 생성한다. 이러한 과정을 반복해 셀프 어텐션된 스코어 맵을 생성한다.

멀티 헤드(Multi-Head)는 이러한 셀프 어텐션을 여러 번 수행해 여러 개의 헤드를 만든다. 각각의 헤드가 독립적으로 어텐션을 수행하고 그 결과를 합친다.

입력받는 $[N, S, d]$ 텐서에서 k 개의 셀프 어텐션 벡터를 생성하고자 한다면 헤드에 대한 차원 축을 생성해 $[N, k, S, d/k]$ 텐서 형태를 구성한다. 이 텐서는 k 개의 셀프 어텐션된 $[N, S, d/k]$ 텐서를 의미한다.

이렇게 생성된 k 개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 **병합(concatenation)**되어 $[N, S, d]$ 의 형태로 출력된다.

덧셈 & 정규화는 멀티 헤드 어텐션을 통과하기 이전의 입력값 $[N, S, d]$ 텐서와 통과한 이후의 출력값 $[N, S, d]$ 텐서를 더함으로써 학습 시 발생하는 기울기 소실을 완화한다. 그리고 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화를 적용한다.

순방향 신경망은 두 가지 방법을 적용할 수 있는데, 선형 임베딩과 ReLU로 이뤄진 인공 신경망이나 1차원 합성곱이 사용된다. 이어서 다시 덧셈 & 정규화 과정이 수행된다.

트랜스포머 인코더는 여러 개의 트랜스포머 인코더 블록으로 구성된다. 이전 블록에서 출력된 벡터는 다음 블록의 입력으로 전달되어 인코더 블록을 통과하면서 점차 입력 시퀀스의 정보가 추상화된다.

마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용되며, 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값 벡터로 활용된다. 이런 방식으로 인코더와 디코더가 서로 정보를 공유한다.

트랜스포머 디코더

트랜스포머 디코더는 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력받는다. 위치 인코딩은 입력 시퀀스 내에서 각 단어의 상대적인 위치 정보를 전달하는 기법이므로 디코더의 입력 임베딩에 위치 정보를 추가함으로써 디코더가 입력 시퀀스의 순서 정보를 학습할 수 있게 된다.

트랜스포머 모델에서 인코더의 멀티 헤드 어텐션 모듈은 **인과성(Casuality)**을 반영한 마스크 멀티 헤드 어텐션 모듈로 대체된다.

인코더의 멀티 헤드 어텐션 모듈과 유사하지만, 마스크 멀티 헤드 어텐션 모듈은 어텐션 스코어 맵을 계산할 때 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라볼 수 있게 마스크를 씌운다. 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌운다.

이러한 마스크를 적용하면 셀프 어텐션에서 현재 위치 이전의 단어들만 참조할 수 있게 되며, 인과성이 보장된다.

마스크 멀티 헤드 어텐션 모듈에서는 마스크 영역에 수치적으로 굉장히 작은 값인 $-\text{inf}$ 마스크를 더해줌으로써 해당 영역의 어텐션 스코어값이 0에 가까워진다.

소프트맥스 계산 시 해당 영역의 어텐션 가중치가 0이 되므로, 마스크 영역 이전에 있는 입력 토큰들에 대한 정보를 참조하지 않게 된다.

이러한 방식으로 마스크 멀티 헤드 어텐션 모듈은 인코더의 멀티 헤드 어텐션 모듈과 유사하지만, 인과성을 보장하면서 셀프 어텐션을 수행할 수 있게 된다.

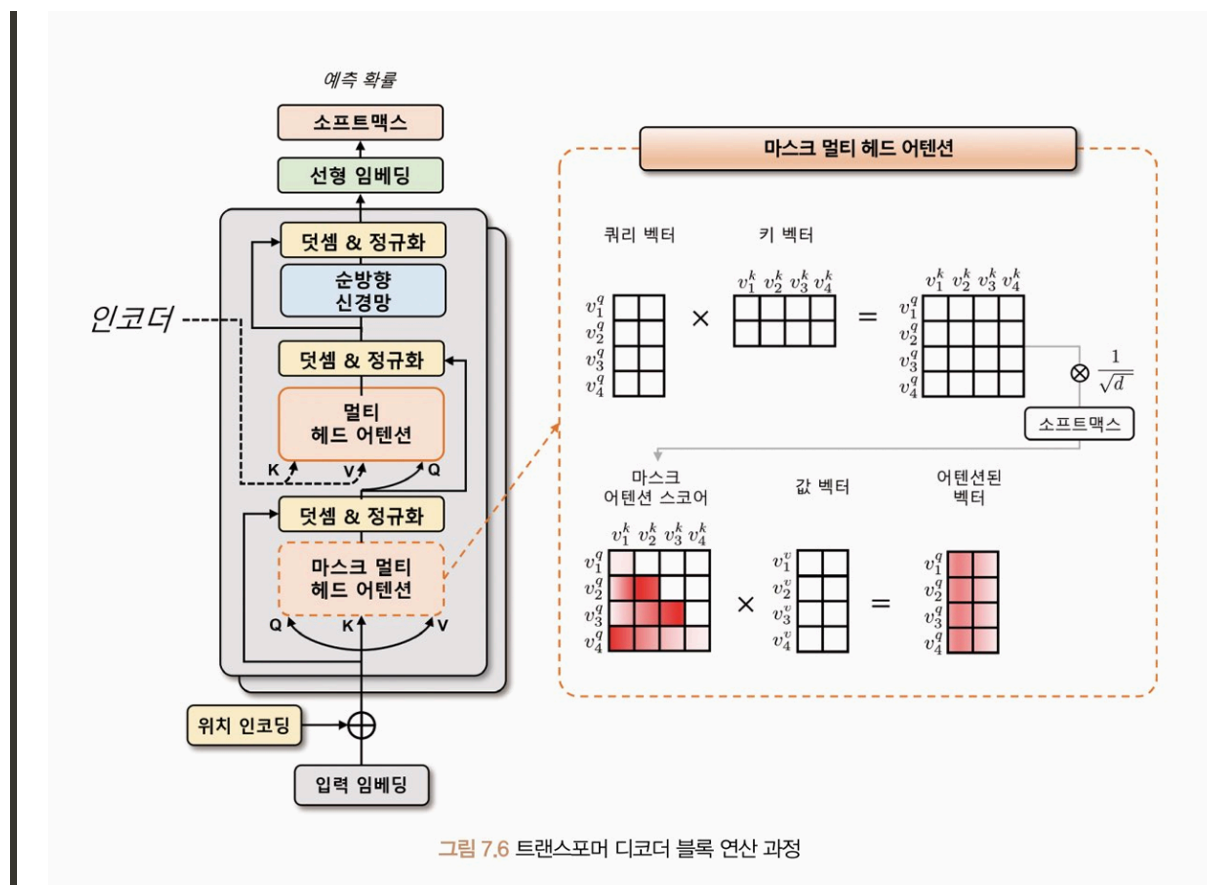


그림 7.6 트랜스포머 디코더 블록 연산 과정

디코더의 멀티 헤드 어텐션에서는 타깃 데이터가 쿼리 벡터로 사용되며, 인코더의 소스 데이터가 키와 값 벡터로 사용된다. 따라서 쿼리 벡터는 타깃 데이터의 위치 정보를 포함한 입력 임베딩과 위치 인코딩을 더한 벡터가 된다.

이후 쿼리 벡터, 키 벡터, 값 벡터를 이용해 어텐션 스코어 맵을 계산하고, 소프트맥스 함수를 적용하여 어텐션 가중치를 구한다. 최종으로 어텐션 가중치와 값 벡터를 가중합하여 멀티 헤드 어텐션의 출력 벡터를 얻을 수 있다.

디코더의 멀티 헤드 어텐션은 인코더의 멀티 헤드 어텐션과 거의 동일하지만, 디코더에서는 셀프 어텐션을 방지하기 위해 마스킹이 적용된다.

트랜스포머의 디코더도 인코더처럼 여러 개의 트랜스포머 디코더 블록으로 구성된다. 이전 트랜스포머 디코더 블록의 산출물은 다음 트랜스포머 디코더 블록의 입력으로 전달된다.

이로써 디코더는 이전 시점에서의 정보를 현재 시점에서 활용할 수 있게 된다. 또한 마지막 디코더 블록의 출력 텐서 $[N, S, d]$ 에 선형 변환 및 소프트맥스 함수를 적용해 각 타깃 시퀀스 위치마다 예측 확률을 계산할 수 있게 된다.

디코더는 타깃 데이터를 추론할 때 토큰 또는 단어를 순차적으로 생성시키는 모델이다. 입력 토큰을 순차적으로 나타내는 방법은 빈 값을 의미하는 PAD 토큰을 사용하는 것이다.

표 7.2 인과성을 고려한 디코더 입력과 출력 예시

추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD], [PAD], [PAD], [PAD]

트랜스포머 모델 하이퍼파라미터

파이토치에서 제공하는 트랜스포머 클래스를 활용하면 트랜스포머 구조의 모델을 쉽게 구현할 수 있다.

- **임베딩 차원(d_model):** 트랜스포머 모델의 입력과 출력 차원의 크기를 정의한다. 이 값은 임베딩 차원 크기와 동일하다.
- **헤드(nhead):** 멀티 헤드 어텐션의 헤드의 개수를 정의한다. 헤드의 개수는 모델이 어텐션을 수행하는 방법을 결정하며, 헤드의 개수가 많을수록 모델의 병렬 처리 능력이 증가한다. 단, 헤드의 개수가 증가할수록 모델 매개변수의 수도 증가한다.
- **인코더 계층 개수(num_encoder_layers)와 디코더 계층 개수(num_decoder_layers):** 인코더와 디코더의 계층 수를 의미하며, 모델의 복잡도와 성능에 영향을 미친다. 계층 개수가 많을수록 더 복잡한 문제를 해결할 수 있지만, 너무 많은 경우 과대적합될 수 있다.
- **순방향 신경망 크기(dim_feedforward):** 순방향 신경망의 은닉층 크기를 정의한다. 순방향 신경망 계층은 트랜스포머 계층의 각 입력 위치에 독립적으로 적용된다. 인코더/디코더 계층과 마찬가지로 모델의 복잡도와 성능에 영향을 미친다.
- **드롭아웃(dropout):** 인코더와 디코더 계층에 적용되는 드롭아웃 비율을 적용하며, **활성화 함수(activation)**는 순방향 신경망에 적용되는 활성화 함수를 의미한다.
- **계층 정규화 입실론(layer_norm_eps):** 계층 정규화를 수행할 때 분모에 더해지는 입실론 값을 정의한다.

트랜스포머 번역 모델

모델 번역 방식은 **그리드 디코딩(Greedy decoding)** 방식으로 번역 결과를 출력한다. 그리드 디코딩은 디코더 네트워크가 생성한 확률 분포에서 가장 높은 확률을 가지는 단어를 선택하는 방법으로, 현재 시점에서 가장 확률이 높은 단어를 선택하여 디코딩을 진행한다.

모델을 추론 방식은 디코더에 참조되는 마지막 인코더 트랜스포머 블록의 벡터(memory), 타겟 데이터의 입력 텐서(ys), 타겟 마스크(target_mask)를 사용한다.

memory 텐서를 생성하려면 소스 문장을 토큰 인덱스로 표현한 source_tensor를 생성하고, source_mask는 소스 문장에서 모든 토큰이 어텐션 될 수 있게 0 값으로 설정한다.

max_len은 num_tokens + 5로 구성되는데, num_tokens는 소스 문장의 토큰 개수를 의미한다. 일반적으로 생성된 문장의 길이가 소스 문장 길이보다 약간 더 길어지는 경우가 많기 때문에 5를 더한다.

더 정확한 번역을 위해 더 많은 학습 데이터를 사용하거나 모델의 구조 또는 하이퍼파라미터 등을 변경해 모델의 성능을 지속적으로 모니터링하고 개선해야 한다.

GPT

GPT(Generative Pre-trained Transformer)는 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델로 가장 널리 알려진 언어 모델이다. 이 모델은 ELMo(Embeddings from Language Model)와 같이 대규모 말뭉치로 사전 학습된 모델로 다양한 다운스트림 작업에서 우수한 성능을 보인다.

GPT 모델은 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델이다. 트랜스포머의 인코더는 입력 문장의 특징을 추출하는 데 초점을 두고 있다면, 디코더는 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장을 생성하는 데 초점을 두고 있다.

이러한 특성 때문에 디코더를 쌓아 모델을 구성하는 것이 자연어 생성과 같은 언어 모델링 작업에서 더 높은 성능을 발휘한다. GPT 모델은 대규모 텍스트 데이터셋으로 사전 학습해 초기화되며, 특정 자연어 처리 작업에 맞게 미세 조정해 사용한다.

GPT는 일반적으로 자연어 생성, 기계 번역, 질의응답, 요약 등 다양한 자연어 처리 작업에 활용되며, 예측 성능이 뛰어나고 적은 데이터로도 높은 성능을 보인다.

이러한 GPT 모델은 다양한 시리즈가 존재한다. 발표 순서대로 GPT-1(2018년), GPT-2(2019년), GPT-3(2020년)이라는 이름으로 알려져 있다. 이 모델들은 거의 동일한 트랜스포머 구조를 사용하며, 버전이 갱신될수록 더 많은 트랜스포머 계층과 데이터를 활용해 학습됐다.

- **GPT-1:** 첫 번째 GPT 모델로, 약 1억 2천만 개의 매개변수가 존재한다.
- **GPT-2:** 두 번째 발표된 모델로, GPT-1의 성능을 크게 능가하며 약 15억 개의 매개변수가 존재한다.
- **GPT-3:** GPT-2보다 훨씬 더 큰 1,750억 개의 매개변수를 갖는 초거대 모델로서 매우 뛰어난 문장 생성 성능을 보여준다. 하지만 GPT-3는 막대한 양의 매개변수를 가진 모델이기 때문에 문장을 생성하는 데 많은 자원을 필요로 한다.

많은 양의 매개변수를 줄이기 위해 OpenAI는 **RLHF(Reinforcement Learning from Human Feedback)** 방법을 도입해 GPT-3.5(InstructGPT) 모델을 학습했다.

RLHF란 인간의 피드백을 이용하는 강화 학습 방법이다. 이 방법은 인간이 모델이 생성한 결과물을 평가하고, 모델이 좋은 결과물을 생성하면 보상을 주어서 모델을 학습시킨다.

이렇게 인간의 지도를 통해 모델을 학습시키는 방법은 기존의 학습 데이터만 사용하는 것보다 더욱 정확하고 효율적인 모델 학습이 가능해 약 60억 개의 가중치로도 뛰어난 성능을 낼 수 있게 됐다.

2022년 10월에는 대화형 질의응답 시스템으로 사용할 수 있는 ChatGPT 서비스가 공개됐다. 이 서비스는 GPT-3.5를 기반으로 하여 자연스러운 대화를 생성하며, 사용자와의 대화를 통해 지속적으로 학습해 발전한다.

또한, 2023년 3월에는 이미지 데이터를 함께 학습하는 GPT-4 모델이 발표됐다. 이 모델은 텍스트 데이터에 국한되지 않고 이미지 데이터도 처리할 수 있게 개선돼 더 복잡한 작업을 수행할 수 있게 발전됐다.

이 외에도 GPT 계열 모델은 GPT-Neo나 GPT-J 등 다양한 시리즈에 모델들이 개발되고 활용되고 있다.

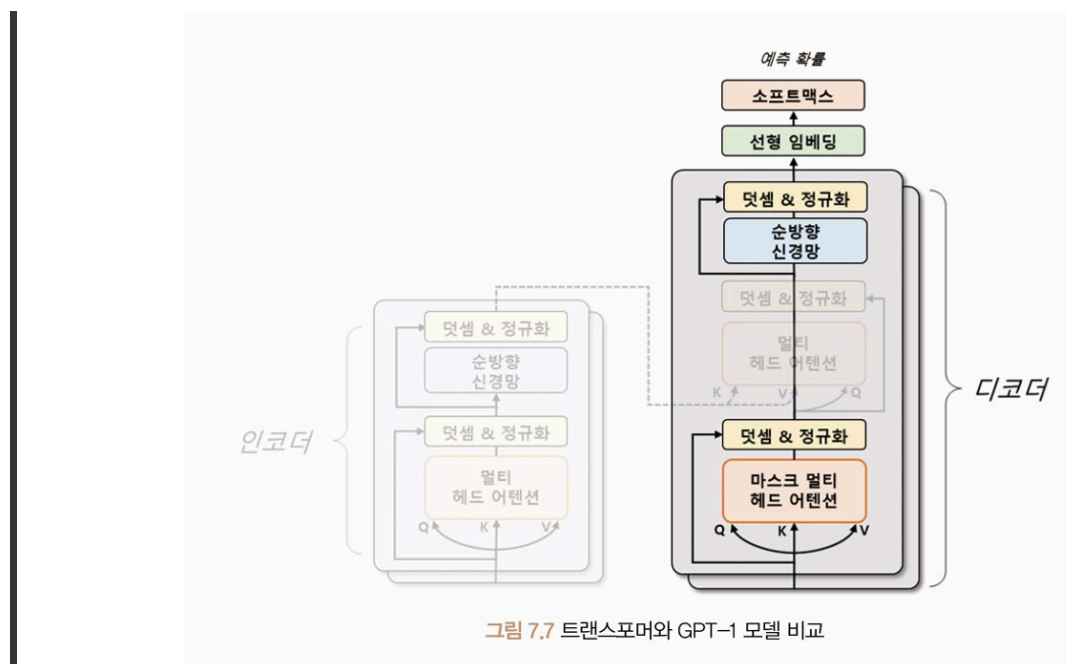
GPT-1

GPT-1은 트랜스포머 구조를 바탕으로 한 **단방향(Unidirectional)** 언어 모델이다. 단방향 언어 모델은 텍스트를 생성할 때 현재 위치에서 이전 단어들에 대한 정보만을 참고해 다음 단어를 예측하는 모델이다.

다시 말해, 이전 단어들을 차례대로 입력하면서 다음 단어를 예측하는 방식으로 동작한다. 이러한 방식은 모델의 계산량이 줄어들기 때문에 학습 속도가 빠르고, 모델의 크기가 작아질 수 있다는 장점이 있다.

GPT-1은 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조를 활용한다. 트랜스포머 구조 중 디코더 부분만 사용하며, 인코더-디코더 간 어텐션 연산을 제외한다.

디코더 부분만을 사용하는 방식으로 모델의 매개변수 수를 줄이면서도 높은 성능을 보일 수 있었다.



GPT-1 모델은 인코더를 사용하지 않고 디코더 부분만 사용하여 모델을 구성하며, 디코더의 두 번째 다중헤드 어텐션은 사용하지 않는다. 12개의 헤드를 가진 트랜스포머의 디코더를 12층 쌓아 모델을 구성하며 약 1억 2천만 개의 매개변수로 학습된다.

GPT-1 모델은 약 4.5GB의 BookCorpus 데이터세트를 사용해 언어 모델을 사전 학습한다. 이때 입력 문장을 임의의 위치까지 보여주고, 뒤에 이어지는 문장을 모델이 자동으로 예측하게 한다.

이 과정에서 별도의 레이블이 필요하지 않기 때문에 비지도 학습으로 학습이 이뤄진다. 별도의 레이블이 없는 비지도 학습이므로 컴퓨팅 자원만 충분하다면 제약 없이 학습할 수 있다는 장점이 있다.

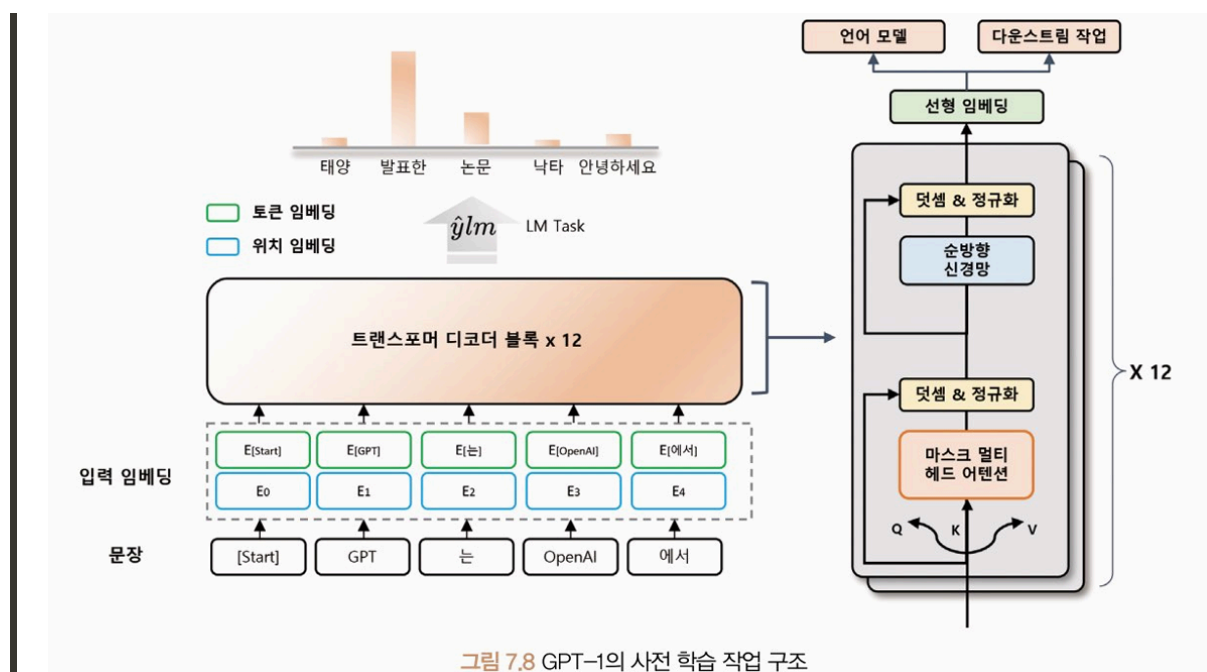


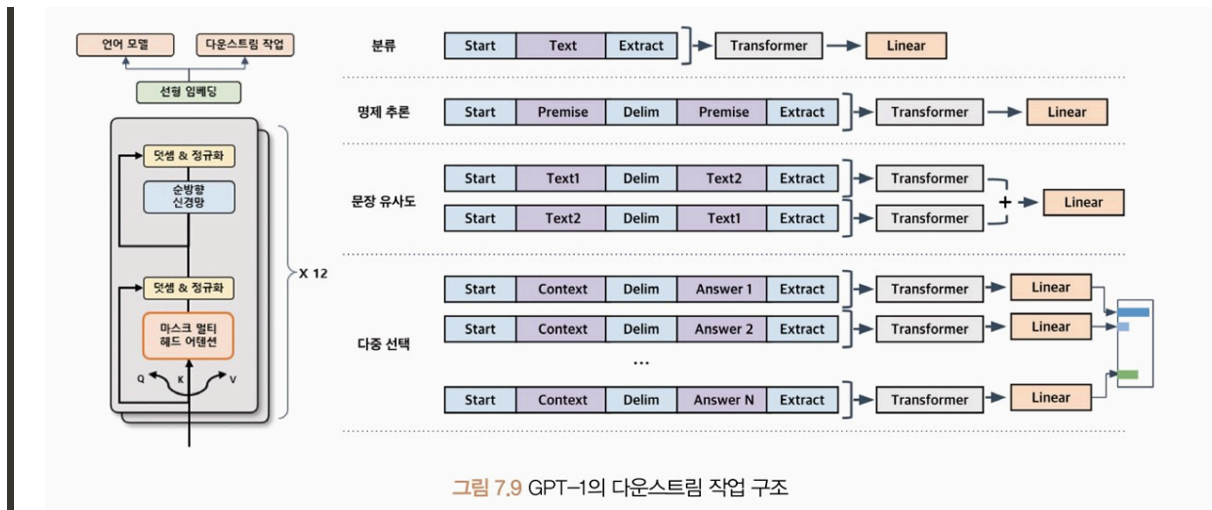
그림 7.8 GPT-1의 사전 학습 작업 구조

GPT-1은 입력 문장의 일부만 보고 다음 토큰을 예측하도록 하는 언어 모델을 통해 사전 학습한다. 이를 위해 입력 문장을 일정한 길이의 블록으로 나누어 처리하고, 각 블록에서는 블록 내의 첫 번째 토큰부터 순서대로 다음 토큰을 예측하게 한다. 이러한 방식으로 나누어 처리하는 방법은 모델이 장기적인 문맥을 이해할 수 있게 한다.

GPT-1의 입력 임베딩은 토큰 임베딩과 위치 임베딩을 이용해 계산된다. 토큰 임베딩은 각 토큰을 고정된 차원의 벡터로 매핑하며, 위치 임베딩은 입력 문장에서 각 토큰의 위치 정보를 나타내는 벡터를 매핑한다.

미세 조정을 위한 다운스트림 작업에서도 GPT-1은 각 작업에 따라 입출력 구조를 바꾸지 않고 언어 모델을 **보조 학습(Auxiliary Learning)**해 사용할 수 있다.

보조 학습이란 모델이 주어진 주요 학습 작업 외에도 다른 작은 부가적인 학습 작업을 동시에 수행하는 것을 의미한다. 보조 작업에서 학습한 정보를 주요 작업에 전달하여 성능을 향상시킬 수 있다.



다운스트림 작업에서는 원하는 목표에 맞게 모델을 세밀하게 조정해야 하기 때문에, 일반적인 언어 모델 학습 시 사용되는 손실 함수 외에 다운스트림 작업에서 필요한 추가 손실 함수가 필요하다.

두 개의 손실 함수를 사용해 최적화하면 보조 학습을 통해 언어 모델을 개선하는 동시에, 다운스트림 작업의 목표를 달성하기 위해 필요한 정보를 학습할 수 있다.

GPT-1 다운스트림 작업에서도 특수 토큰이 사용된다. GPT-1은 문장의 시작을 의미하는 `<start>` 토큰, 두 문장의 경계를 의미하는 `<delim>` 토큰, 문장의 마지막을 의미하는 `<extract>` 토큰을 사용한다.

GPT-2

또한 GPT-1은 입력 문장을 최대 512의 길이만큼 받아들여 다음 토큰을 예측했지만, GPT-2에서는 1,024의 길이까지 입력을 처리할 수 있다. 이외에도 12개의 디코더 층을 사용한 GPT-1과는 달리, 48개의 디코더 계층을 사용해 이전 모델인 GPT-1보다 더욱 복잡한 패턴을 학습할 수 있게 됐다. 이전 모델인 GPT-1은 1,200만 개의 매개변수를 사용해 언어 모델링 작업을 수행했지만, GPT-2에서는 15억 개의 매개변수를 갖게 됐다.

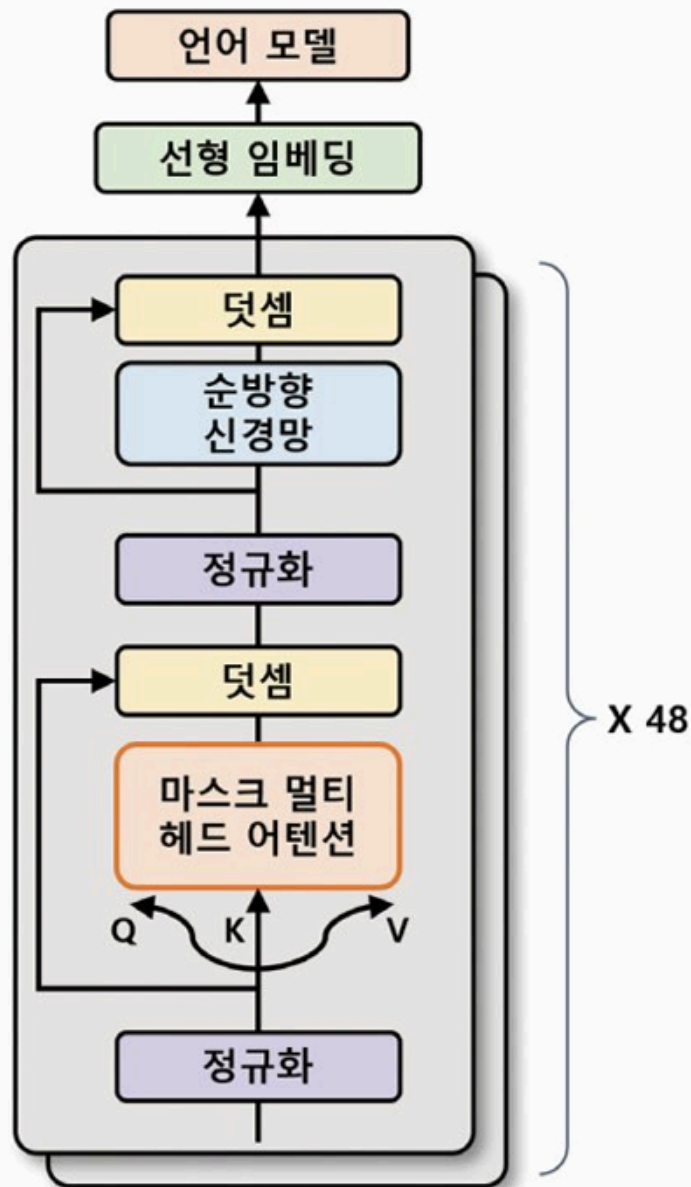


그림 7.10 GPT-2의 모델 구조

GPT-2는 GPT-1보다 훨씬 더 많은 양의 데이터를 이용하여 사전 학습됐다. GPT-2는 미국의 웹사이트에서 스크래핑한 약 8백만 개의 문서를 사용해 약 40GB 데이터로 사전 학습을 수행했다.

더 많은 데이터로 학습된 GPT-2는 더욱 자연스러운 문장 생성 능력을 지니고 있으며, 더 다양한 분야에서 활용될 수 있다.

또한, GPT-2는 제로-샷 학습이 가능하다. GPT-2는 별도의 미세 조정 없이 다운스트림 작업에서도 사용할 수 있게 사전 학습 과정에서 특정한 형식의 데이터를 입력한다.

GPT-3

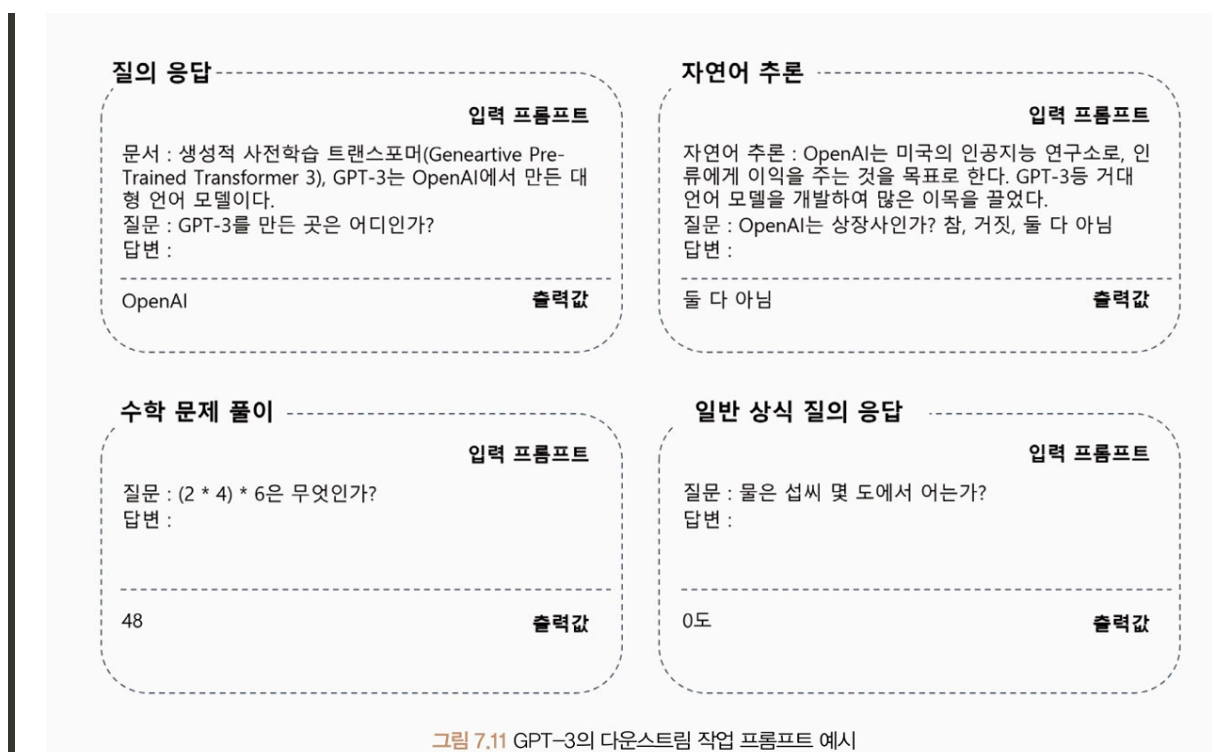
GPT-1과 GPT-2의 모델 구조가 큰 차이 없었듯이, GPT-3도 GPT-2와 거의 동일한 모델 구조를 가지고 있다. 그러나 GPT-3는 몇몇 모델 매개변수의 변화를 통해 GPT-2보다 약 116배 많은 매개변수를 가진 모델이 됐다.

GPT-3는 매우 규모가 큰 모델로 96개의 헤드를 가진 멀티 헤드 어텐션을 사용하며, 트랜스포머 디코더 층도 96개의 층을 사용한다. 멀티 헤드 어텐션 과정에서 연산량을 줄이기 위해 **희소 어텐션(Sparse Attention)**과 일반 어텐션을 섞어 사용한다.

이에 더해 토큰 임베딩의 크기도 1,600에서 12,888로 대폭 증가한다. GPT-2가 최대 1,024개의 토큰까지 입력할 수 있었다면, GPT-3는 2,048개까지 입력할 수 있어 더욱 긴 문장에 대한 처리 능력이 향상됐다.

GPT-3는 웹 크롤링, 위키백과, 서적 등에서 수집한 약 45TB에 달하는 방대한 양의 데이터 세트를 이용하여 모델을 학습한다. 이 모델은 다운스트림 작업으로 학습하지 않았지만, 질의 응답, 번역, 요약, 문서 생성 등 다양한 다운스트림 작업에서도 높은 성능을 보인다.

GPT-3도 GPT-2와 같이 **프롬프트(prompt)** 형식으로 작업을 수행한다.



GPT-3는 새로운 도메인에서의 작업에 대해서도 높은 일반화 능력을 보이며, 질의응답, 수학 문제 풀이, 자연어 추론 등과 같은 기존 자연어 처리 분야에서는 이전의 기술적 한계를 극복하는 성과를 보여준다.

하지만 GPT-3는 큰 모델이기 때문에 매우 느리고 비용이 많이 들어 일반 사용자나 소규모 기업에서 활용하기 어려우며, 사전 학습된 데이터에 기반하여 작동하기 때문에 데이터가 편

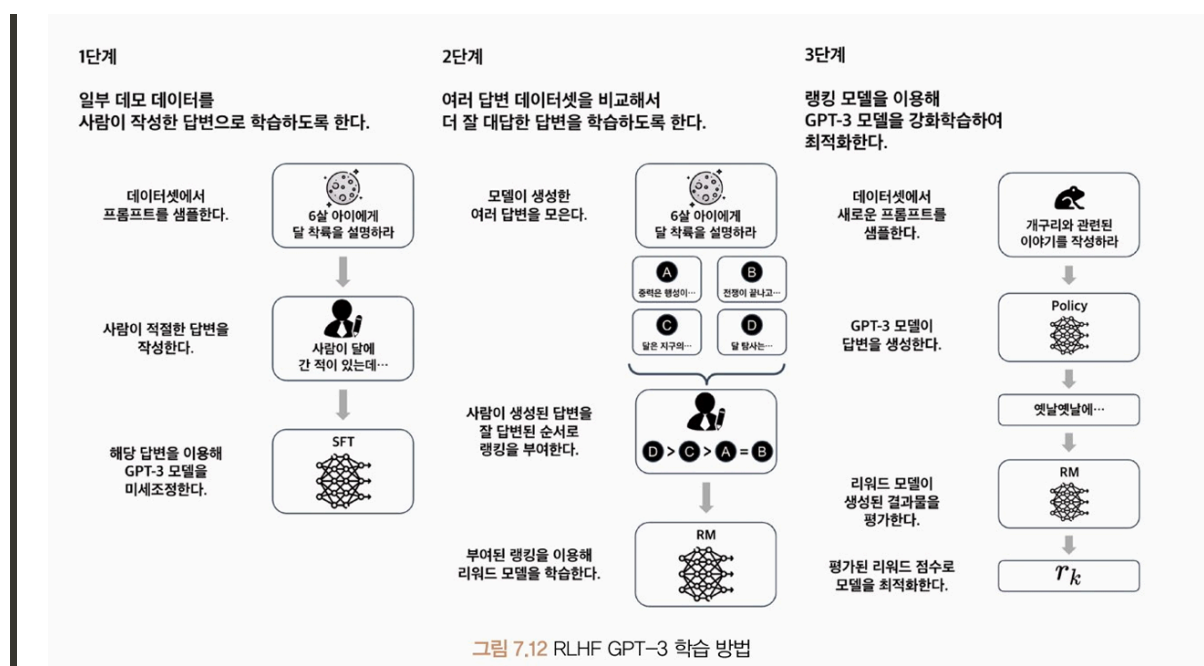
향돼 있거나 정확하지 않을 경우 오류가 발생할 수 있다.

GPT-3는 자연어 처리 작업에 대한 일반화 능력이 높지만, 사람과 비교할 때 인간적인 이해력과 상식적인 추론 능력이 부족하다. 모델이 생성한 텍스트는 항상 사실일 필요가 없으며, 편견이나 혐오 표현 등이 포함될 수도 있다.

이러한 결과물을 사용할 때는 항상 검토하고 검증하는 과정이 필요하다. 또한, 생성된 텍스트의 저작권은 모델을 학습시킨 데이터와 모델을 만든 회사에 있으므로 저작권 문제에 주의해야 한다.

GPT 3.5

InstructGPT라고도 불리는 GPT-3.5는 GPT-3의 모델 구조를 그대로 따르면서도, 새로운 학습 방법인 RLHF를 도입해 모델의 매개변수 수를 줄이고 모델의 자연스러움을 높인 것이 특징이다.



RLHF의 학습 방법은 데이터세트에서 임의의 프롬프트를 가져온다. 그리고 이 프롬프트에 대해 사람이 직접 적절한 답을 작성하게 한다.

이때 작성된 답변은 모델이 생성한 문장과 함께 평가해 모델이 생성한 문장이 자연스러우면 보상받고, 그렇지 않으면 보상받지 못한다. 이러한 보상을 바탕으로 강화 학습을 통해 모델을 학습한다. 이러한 과정을 **지도 미세 조정(Supervised Fine-Tuning, SFT)**이라 한다.

RLHF를 통해 학습한 모델은 GPT-3 모델보다 더 자연스러운 문장을 생성할 수 있다. 하지만 학습 데이터양을 고려하면 전체 프롬프트에 사람이 답변을 다는 것은 현실적으로 어렵다. 그러므로 GPT 모델이 얼마나 잘 답변했는지 평가하는 보상(Reward) 모델을 학습한다.

GPT 모델은 시드(seed)에 따라 다른 답변을 생성하므로 내용이 서로 다른 여러 개의 답변이 생성될 수 있다.

보상 모델로 사람이 평가하고 그에 따른 랭킹을 부여한다. 사람이 부여한 랭킹은 생성된 답변의 우수성을 평가한 결과이며, 이를 보상 모델의 입력으로 사용한다. 보상 모델은 이렇게 생성된 답변과 랭킹 정보를 활용하여 학습하며, 이를 통해 모델은 더 자연스러운 문장을 생성할 수 있게 된다.

RLHF에서는 주어진 프롬프트(State)에 대해 GPT-3 언어 모델(Policy)이 답변을 생성(Action)하고, 이를 이용해 사람의 피드백을 기반으로 한 학습된 보상 모델을 평가(Reward)한다.

이 과정에서 생성된 답변과 랭킹 정보를 활용하여 리워드 모델이 학습되며, 이를 통해 더욱 자연스러운 문장 생성이 가능해지는 GPT-3.5(InstructGPT)가 학습된다. 이 과정을 반복함으로써 더욱 정교한 답변 생성 모델을 학습할 수 있다.

GPT-4는 이전 모델과 비교했을 때 높은 성능 향상을 보였지만, 언어 모델이 일부 입력에 대해 현실적이지 않은 결과를 생성하는 **환각(Hallucination)** 현상이 아직 존재한다. 환각 현상이란 모델이 인간과 같은 추론과 판단 능력을 갖추지 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상을 말한다.

GPT-4

GPT-4는 이전의 GPT 모델과는 달리, 텍스트 데이터뿐만 아니라 이미지 데이터도 인식 가능한 **멀티모달(Multimodal)** 모델이다.

GPT-3.5가 입력으로 최대 4,096개의 토큰을 처리할 수 있었던 반면, GPT-4는 최대 여덟 배인 32,768개의 토큰을 입력받을 수 있다. 즉, GPT-3.5에서 3,000개의 단어를 처리할 수 있었다면 GPT-4는 25,000개의 단어를 처리할 수 있게 됐다.

대규모 다중 작업 언어 이해(MMLU) 벤치마크 테스트 결과, GPT-3.5에서는 약 70%의 성능을 보였지만, GPT-4에서는 약 85%까지 성능이 향상됐다. 한국어로 번역한 MMLU 벤치마크에서 77%를 기록하며 GPT-3.5의 영어 성능보다 높은 성능을 내는 것으로 알려졌다.

또한, GPT-3.5는 미국 변호사 시험에서 213점을 받았지만, GPT-4는 298점으로 상위 10%의 점수를 받았으며, SAT 읽기와 쓰기 시험에서는 GPT 3.5가 받은 670점보다 40점 높은 710점을 받아 상위 7%의 점수를 받았다.

GPT-4는 모델의 매개변수 수나 학습 데이터 및 방법은 공개되지 않았지만, ChatGPT Plus나 마이크로소프트 Bing을 통해 모델을 사용할 수 있다.

BERT

BERT(Bidirectional Encoder Representations from Transformers)는 2018년 구글에서 발표한 언어 모델로 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델이다.

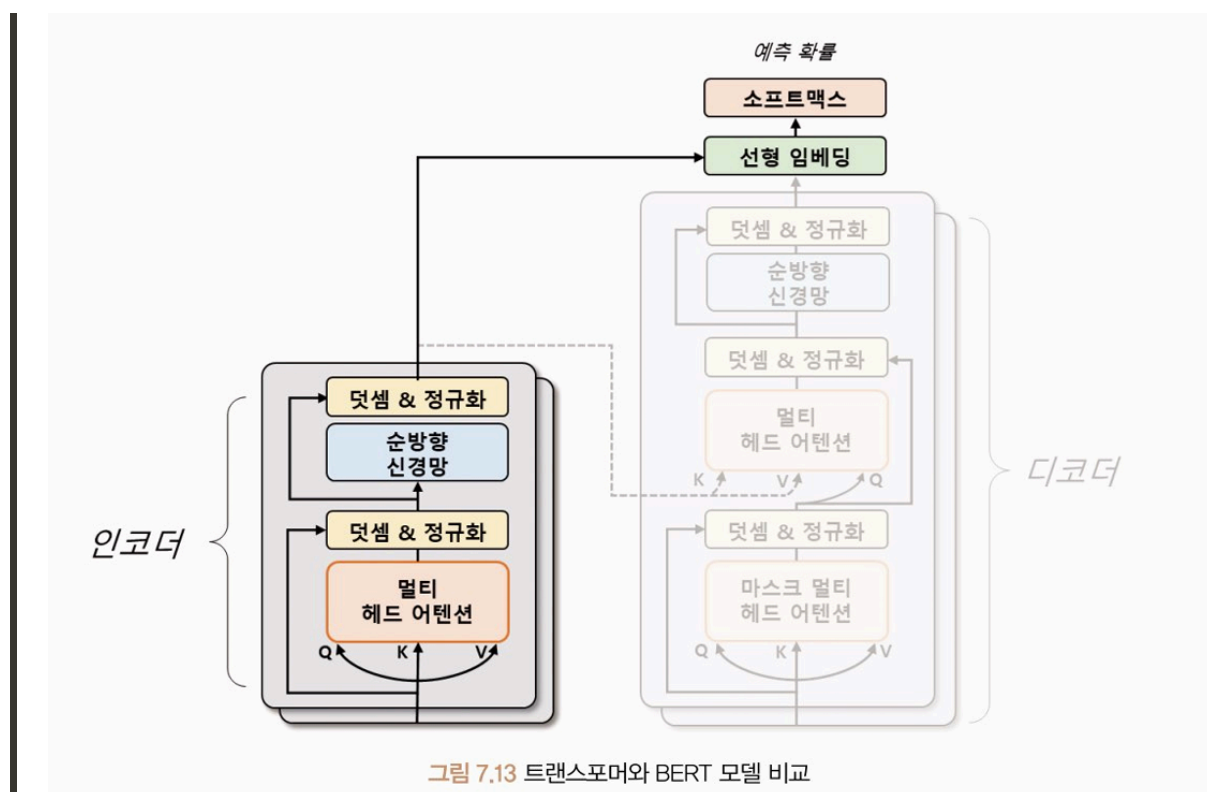
양방향(Bidirectional) 인코더는 입력 시퀀스를 양쪽 방향에서 처리하여 이전과 이후의 단어를 모두 참조하면서 단어의 의미와 문맥을 파악한다. 기존의 언어 모델은 문장을 좌에서 우로 순차적으로 학습해 단어의 의미와 문맥을 파악할 때 이전 단어만 고려할 수밖에 없었다.

하지만 BERT는 양방향으로 문장을 학습하므로 이전과 이후의 단어를 모두 참조해 단어의 의미와 문맥을 파악할 수 있다. 이를 통해 BERT는 기존 모델들보다 더 정확하게 문맥을 파악하고, 다양한 자연어 처리 작업에서 높은 성능을 보인다.

BERT는 대규모 데이터를 사용해 사전 학습되어 있으므로 전이 학습에 주로 활용된다.

BERT는 일부 또는 전체를 다른 작업에서 재사용해 적은 양의 데이터로도 높은 정확도를 달성할 수 있으며, 모델 학습 시간을 크게 단축할 수 있다.

BERT 모델은 트랜스포머의 인코더를 기반으로 하며, 이를 효과적으로 재활용함으로써 학습 데이터의 양과 다양성을 고려한 높은 성능을 보인다.



BERT는 입력 문장의 의미와 구조를 학습하고 다양한 자연어 처리 작업에 적용할 수 있는 사전 학습된 언어 모델이다. BERT는 트랜스포머의 인코더 모듈만을 사용해 입력 문장을 처리한다.

인코더는 입력 문장의 단어들을 임베딩해 각 단어의 의미를 벡터화한 후, 이를 순차적으로 처리하여 문장 전체의 의미를 추출한다. BERT는 입력 문장의 단어를 좌우 양방향으로 처리해 문맥 정보를 모델링하므로 인코더 계층만을 사용해 학습한다.

BERT는 사전 학습을 위해 **마스킹된 언어 모델링(MLM)**과 **다음 문장 예측 방법(NSP)**을 사용한다.

BERT 사전 학습 방법

마스킹된 언어 모델링(Masked Language Modeling, MLM)은 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방식이다. 예를 들어 'I'm learning PyTorch'라는 문장에서 'learning'을 마스킹하면, 'I'm [MASK] PyTorch'가 되어 BERT는 [MASK]에 들어갈 단어를 예측한다. 이 과정에서 양방향 문맥 정보를 참고한다.

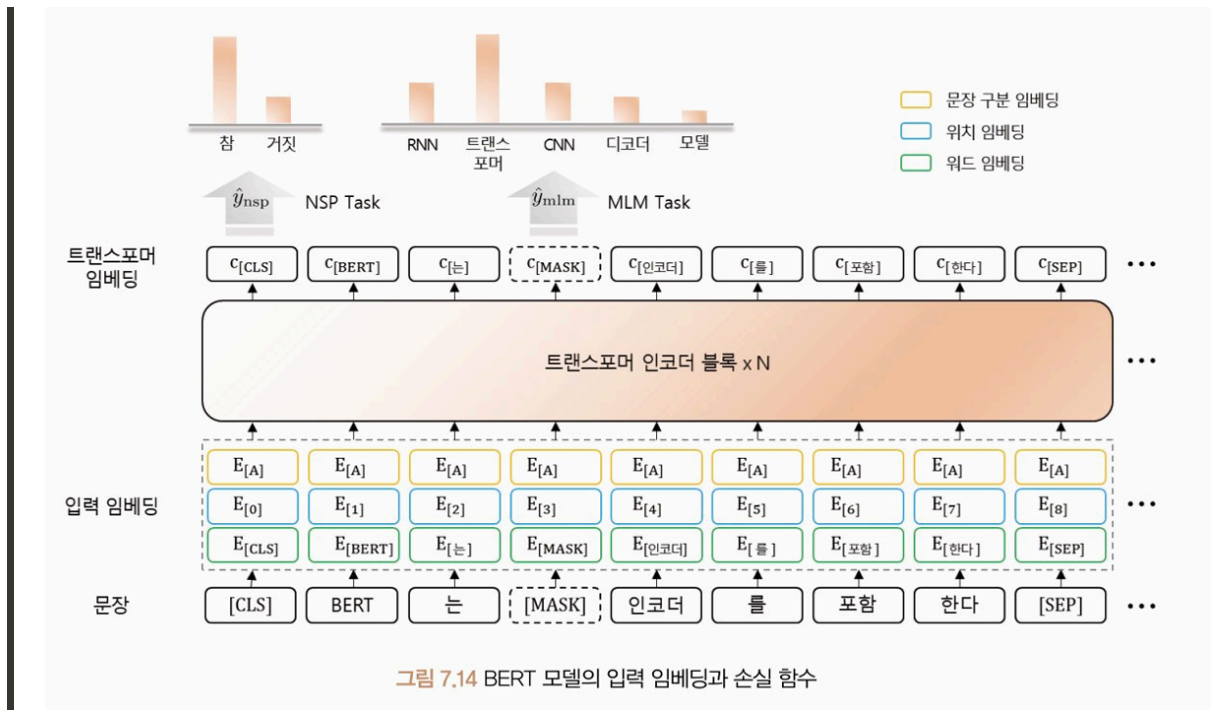
BERT는 양방향으로 문장을 학습해 문맥 정보를 모델링하므로 입력 문장에서 누락된 단어를 추론하는 능력을 갖게 된다. 이를 통해 BERT는 문장 전체 의미를 이해하는 능력이 향상됐다.

다음 문장 예측(Next Sentence Prediction, NSP)은 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업이다. 이 과정에서 BERT는 두 문장 간의 관계를 학습하고, 문장 간의 의미적인 유사성을 파악한다.

BERT 모델은 입력 문장에 특수한 토큰들을 추가해 모델이 학습하고 추론하는 과정에서 필요한 정보를 제공한다. BERT에서는 [CLS] 토큰, [SEP] 토큰, [MASK] 토큰이 사용된다.

- **[CLS] 토큰**: 입력 문장의 시작 부분에 추가되는 토큰이며, 이 토큰을 이용해 문장 분류 작업을 위한 정보를 제공한다. 이를 통해 모델은 입력 문장이 어떤 유형의 문장인지 미리 파악할 수 있게 된다.
- **[SEP] 토큰**: 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용되는 토큰이다.
- **[MASK] 토큰**: 입력 문장 내에서 임의로 선택된 단어를 가리키는 특별한 토큰이다. 이 토큰은 주어진 문장에서 일부 단어를 가린 후 모델의 학습과 예측에 활용한다.

따라서 BERT 모델의 입력 시퀀스는 **[CLS] 문장-1 [SEP] 문장-2 [MASK] [SEP]** 등의 구조를 가질 수 있다.



[MASK] 토큰 적용 방법은 텍스트 토큰 중 15%에 해당하는 단어를 대상으로 마스킹을 수행한다. 이 중 80%는 [MASK] 토큰으로 대체하고, 10%는 어휘 사전에 존재하는 무작위 단어로 변경한다. 그리고 나머지 10%는 실제 토큰을 그대로 사용한다.

BERT는 사전 학습을 수행한 후, 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용할 수 있다. 예를 들어 문장 분류, 감성 분석, 질문 응답, 기계 번역 등 다양한 작업이 가능하다. 미세 조정 과정에서는 해당 작업에 맞는 계층을 추가하고 손실 함수를 정의해 학습한다.

분류 문제에서는 일반적으로 [CLS] 토큰 벡터를 사용하고, 각 토큰마다 예측이 필요한 경우에는 모든 토큰 벡터를 사용한다.

BART

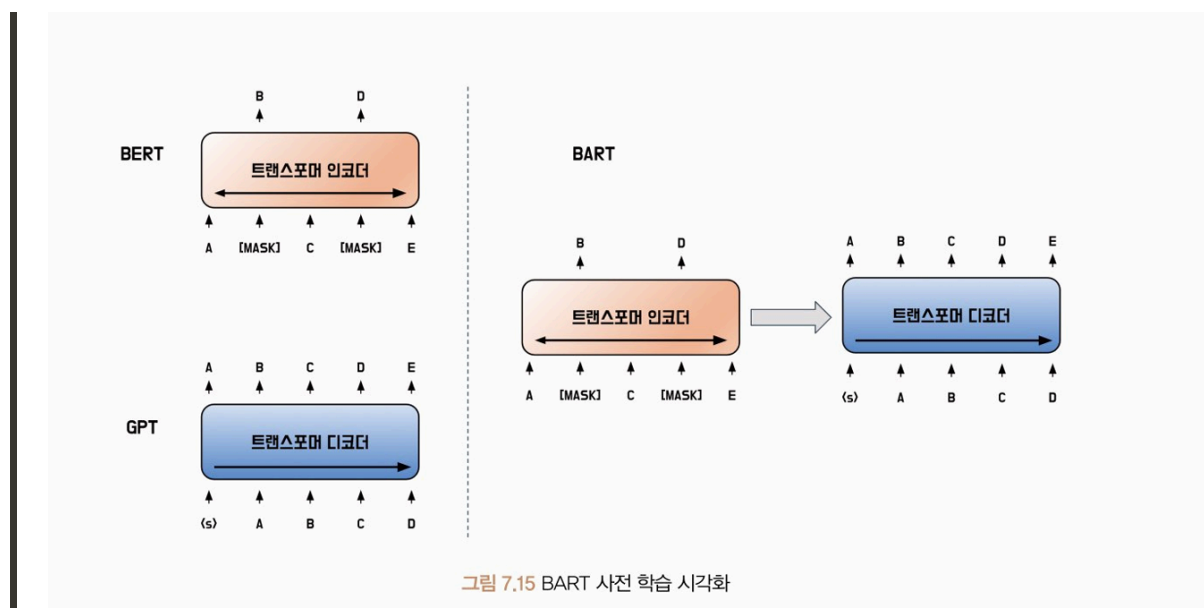
BART(Bidirectional Auto-Regressive Transformer)는 2019년 메타의 FAIR 연구소에서 발표한 트랜스포머 기반의 모델이다.

BART는 BERT의 인코더와 GPT의 디코더를 결합한 **시퀀스-시퀀스(Sequence-to-Sequence, Seq2Seq)** 구조로 **노이즈 제거 오토인코더(Denoising Autoencoder)**로 사전 학습된다. 오토인코더 사전 학습 방법은 입력 데이터에 잡음을 추가하고, 잡음이 없는 원본 데이터를 복원하도록 학습하는 방식으로 수행된다.

BERT의 인코더는 입력 문장에서 일부 단어를 무작위로 마스킹해 처리하고, 마스킹된 단어를 맞추도록 학습한다. BERT의 인코더는 문장 전체의 맥락을 이해하고 문맥 내 단어 간 상호작용을 파악할 수 있다.

반면에 GPT는 언어 모델을 이용해 문장의 이전 토큰들을 입력으로 받고, 다음에 올 토큰을 맞추도록 학습한다. 이를 통해 GPT는 문장 내 단어들의 순서와 문맥을 파악하며, 다음에 올 단어를 예측하는 능력을 갖게 된다.

BART는 사전 학습 시 BERT의 인코더와 GPT의 디코더가 학습하는 방법을 일반화해 학습한다.



BART는 사전 학습 시 노이즈 제거 오토인코더를 사용하므로 입력 문장에 임의로 노이즈를 추가하고 원래 문장을 복원하도록 학습한다.

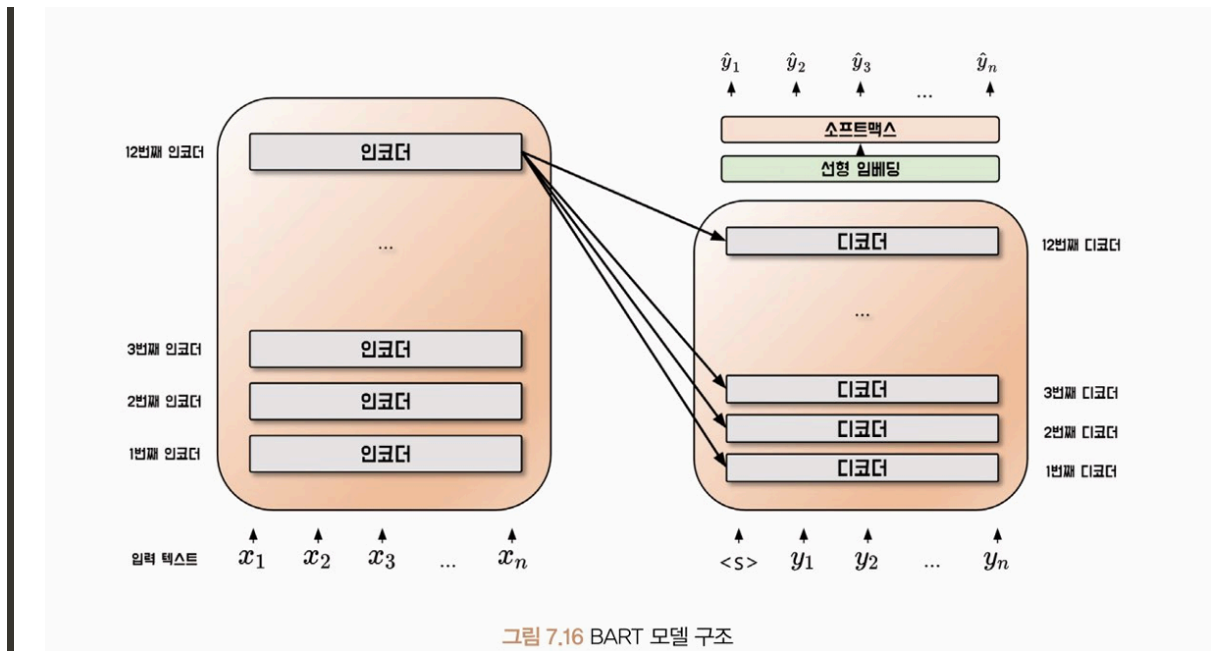
노이즈가 추가된 텍스트를 인코더에 입력하고 원본 텍스트를 디코더에 입력해 디코더가 원본 텍스트를 생성할 수 있게 학습하는 방식이다.

이는 BERT가 인코더 학습 시 텍스트에 마스킹을 추가해 노이즈를 주는 방법과 GPT가 디코더 학습 시 다음에 등장할 단어를 예측하는 방법으로 간주할 수 있다.

이를 통해 BART는 문장 구조와 의미를 보존하면서 다양한 변형을 학습할 수 있다. 이 구조는 입력 문장에 큰 제약 없이 노이즈 기법을 적용할 수 있으므로 더 풍부한 언어적 지식을 습득할 수 있게 한다.

인코더를 사용함으로써 순방향 정보만 인식할 수 있는 GPT의 단점을 개선해 양방향 문맥 정보를 반영할 수 있으며, 디코더를 사용함으로써 문장 생성 분야에서 뛰어나지 않았던 BERT의 단점을 개선했다.

BERT와 BART의 차이점은 BERT는 인코더만 사용하고, BART는 인코더와 디코더를 모두 사용한다는 것이다. BART가 문장을 생성하거나 번역하는 작업에 적합하며, 문장 독해, 문서 요약, 질의응답 등에 있어 높은 성능을 보인다.



BART는 인코더와 디코더를 모두 사용하므로 트랜스포머와 유사한 구조를 가지고 있지만, 차이점이 존재한다. 트랜스포머에서는 인코더의 모든 계층과 디코더의 모든 계층 사이의 어텐션 연산을 수행했다면, BART는 인코더의 마지막 계층과 디코더의 각 계층 사이에만 어텐션 연산을 수행한다.

BART에서는 인코더의 마지막 계층과 디코더의 각 계층 사이에서만 어텐션 연산을 수행하므로 정보 전달을 최적화하고 메모리 사용량을 줄일 수 있다.

BART 사전 학습 방법

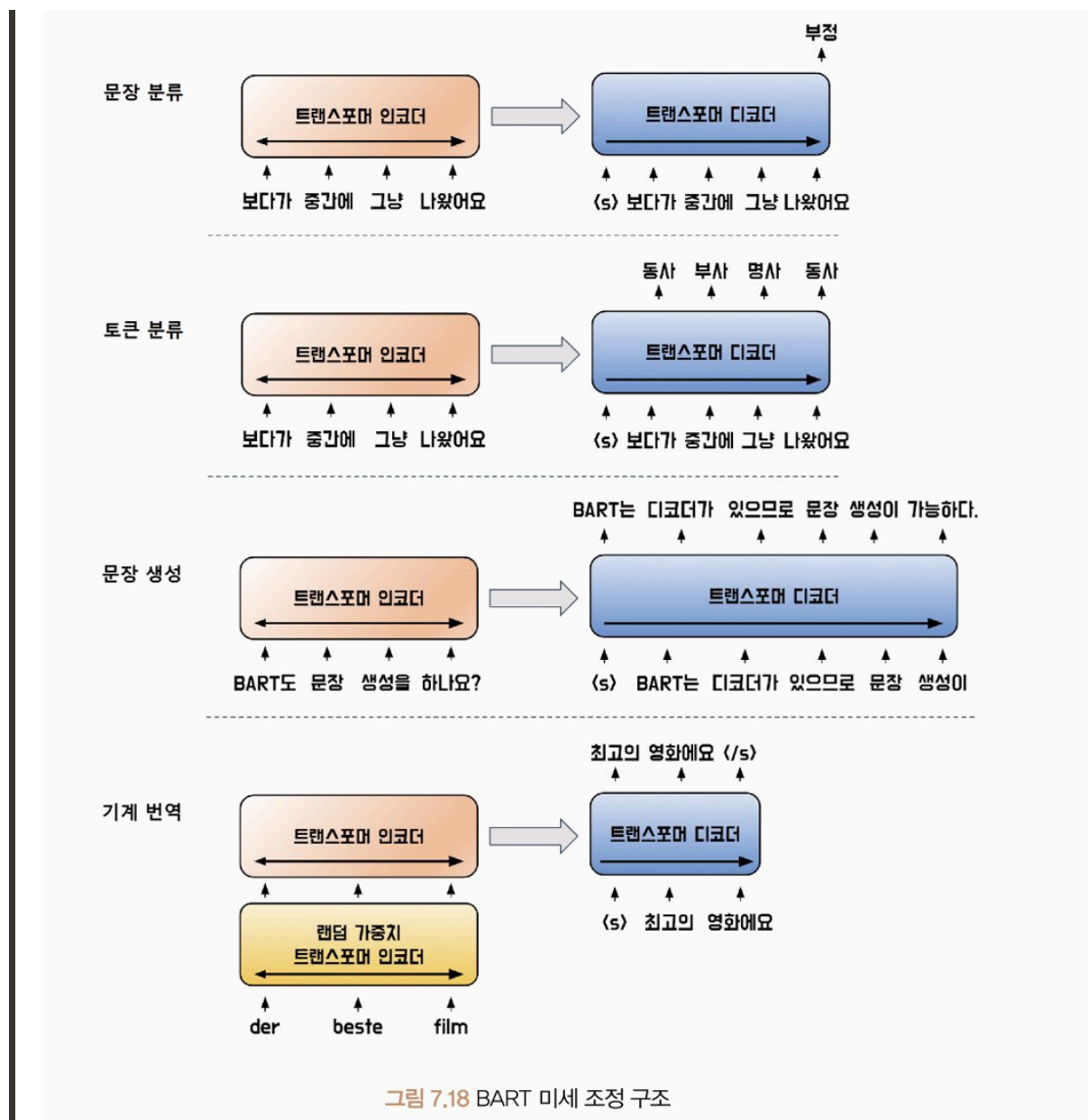
BART의 인코더는 BERT의 마스킹된 언어 모델링(MLM) 이외에도 다양한 노이즈 기법을 사용한다.



- **토큰 마스크(Token Masking):** BERT에서 사용한 MLM과 동일한 기법으로, 입력 문장의 일부 토큰을 마스크 토큰으로 치환하는 방법이다. 이 기법을 이용하면 문장 내에서 어떤 단어가 중요한 역할을 하는지 학습할 수 있으며, 입력 문장의 문맥을 이해하고 문장 내에서 중요한 정보를 추출하는 능력이 향상된다.
- **토큰 삭제(Token Deletion):** 입력 문장의 일부 토큰을 치환하는 것이 아니라, 삭제하는 방법이다. 토큰 마스크와 다르게 어떤 위치의 토큰이 삭제되었는지도 맞춰야 한다. 이 기법을 이용하면 입력 문장에서 불필요한 정보나 중요하지 않은 정보를 자동으로 필터링해 처리할 수 있게 되며, 모델의 학습과 예측 시간을 줄이고, 모델의 일반화 성능을 향상시킬 수 있다.
- **문장 순열(Sentence Permutation):** 마침표(.)를 기준으로 문장을 나눈 뒤, 문장의 순서를 섞는 방법이다. 모델은 원래의 문장 순서를 맞춰야 한다. 이 기법은 문장 내에서 단어들 어떻게 연결되어 있는지 더 잘 파악하게 되며 입력 문장이 다른 순서로 주어졌을 때에도 잘 처리할 수 있게 된다.
- **문서 회전(Document Rotation):** 임의의 토큰으로 문서가 시작하도록 한다. 문장 순열과는 다르게 문장의 순서는 유지한다. 모델은 문서의 원래의 시작 토큰을 맞춰야 한다. 이 기법은 모델이 문서의 시작점을 인식할 수 있도록 한다.
- **텍스트 채우기(Text Infilling):** 몇 개의 토큰을 하나의 구간(span)으로 묶고 일부 구간을 마스크 토큰으로 대체한다. 묶은 구간의 길이(개수)는 0부터 임의의 값까지 설정할 수 있으며, 일반적으로 포아송 분포로 구간을 나누며, λ 를 3으로 설정해 계산한다. 구간의 길이가 0인 경우는 해당 위치에 변환된 토큰이 없음을 의미한다. 모델은 연속된 마스크 토큰을 복구하되, 실제로는 마스크되지 않은 토큰도 구분해야 한다.

BART 미세 조정 방법

BART는 인코더와 디코더를 모두 사용하는 구조를 가지고 있기 때문에 미세 조정 시 각 다운스트림 작업에 맞게 입력 문장을 구성해야 한다. 즉, 인코더와 디코더에 다른 문장 구조로 입력한다.



문장 분류 작업에서는 입력 문장을 인코더와 디코더에 동일하게 입력하고 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용한다. 이때 BERT의 CLS 토큰과 비슷하지만, BART는 전체 입력과 어텐션 연산이 적용된 은닉 상태를 사용한다.

토큰 분류 작업에서도 BART는 입력 문장을 인코더와 디코더에 동일하게 입력한다. 디코더의 각 시점별 마지막 은닉 상태를 토큰 분류기의 입력값으로 사용한다. 전체 입력과 디코더의 각 시점별 은닉 상태와의 어텐션 연산을 수행한다.

BART는 트랜스포머 디코더를 사용하기 때문에 BERT가 해결하지 못했던 문장 생성 작업을 수행할 수 있다. 특히 입력값을 조작해 출력을 생성하는 **추상적 질의응답(Abstractive Question Answering)**과 **문장 요약(Summarization)**과 같은 작업에 적합하다.

이러한 작업들은 BART의 사전 학습 방식과 유사하기 때문에 뛰어난 성능을 보인다. 이러한 학습 방법으로 BART는 문장의 의미를 파악하고, 이를 기반으로 새로운 문장을 생성하는 능력을 갖게 된다.

BART는 문장 내부의 토큰 사이의 상관관계를 파악해 문장의 의미를 더욱 정확하게 이해할 수 있다. 입력 문장의 전체적인 의미를 고려하므로 BART가 자연어 생성 분야에서 매우 효과적인 모델임을 보여준다.

ELECTRA

ELECTRA(Efficiently Learning an Encoder that Classifies Token Replacements Accurately)는 2020년 구글에서 발표한 트랜스포머 기반의 모델이다.

BERT를 비롯해 많은 자연어 처리 모델은 마스킹된 언어 모델링(MLM) 방식을 사용하거나 일부 토큰을 [MASK]로 대체하여 입력을 손상시키고 원래 토큰을 복원하는 모델을 학습한다.

하지만 ELECTRA는 입력을 마스킹하는 대신 **생성자(Generator)**와 **판별자(Discriminator)**를 사용해 사전 학습을 수행한다. ELECTRA의 사전 학습 접근 방식은 변형기 모델인 생성자와 판별자를 학습하므로 생성적 적대 신경망(GAN)과 유사한 방법으로 학습이 수행된다.

생성 모델은 실제 데이터와 비슷하게 토큰을 생성해 다른 토큰으로 대체하고 판별 모델이 생성 모델이 만든 데이터와 실제 데이터를 입력받아 어떤 데이터가 실제 데이터인지, 아니면 생성된 데이터인지 구분한다.

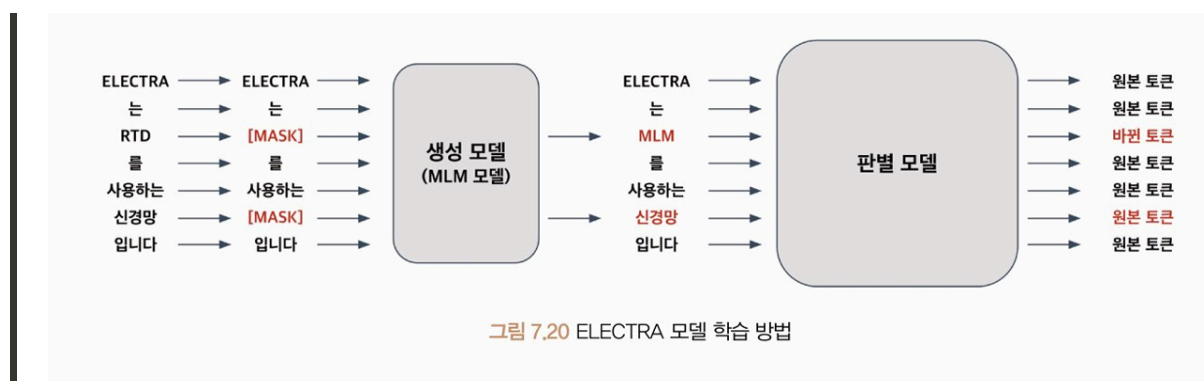
ELECTRA는 GAN을 사용해 학습하므로 이전 언어 모델과 비교하여 더 효율적인 학습이 가능하다. 덕분에 대규모 데이터셋에서 모델을 더 빠르게 학습할 수 있다. 생성 모델을 통해 토큰을 생성하므로 다양한 자연어 생성 작업에서 보다 자연스러운 문장을 생성하게 된다.

또한 BERT와 같은 모델과 비교한다면 모델의 매개변수 수가 더 적다. 그로 인해 모델의 크기가 줄어들어 모델을 더 빠르게 실행할 수 있으며, 더 적은 메모리를 사용한다.

ELECTRA 사전 학습 방법

ELECTRA의 생성자 모델과 판별자 모델은 모두 트랜스포머 인코더 구조를 따른다. 생성자 모델은 입력 문장의 일부 토큰을 마스크 처리하고 마스크 처리된 토큰이 원래 어떤 토큰이었는지 예측하며 학습한다.

반면에 판별자 모델은 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 인해 바뀐 토큰인지 맞히며 학습한다. 이러한 학습 방법을 **RTD(Replaced Token Detection)**라고 한다.



생성자 모델은 BERT의 마스킹된 언어 모델과 동일하다. 입력 문장의 15%를 마스크 처리해 마스크 처리된 토큰이 원래 어떤 토큰이었는지를 맞히며 학습한다.

판별자 모델은 생성자 모델이 복원한 결괏값을 입력받아 각 토큰이 원본 토큰인지 바뀐 토큰인지를 판별한다.

이 과정은 생성자와 판별자로 구성된 생성적 적대 신경망(GAN)과 유사하지만, 몇 가지 차이점이 있다. 먼저, 생성자 모델이 원래 토큰을 정확히 예측한 경우 이 토큰은 생성된 토큰이 아닌 원본 토큰으로 인식한다.

두 번째로 GAN은 생성 모델과 판별 모델을 적대적으로 학습한다. 생성 모델은 판별 모델이 실제 데이터와 구분하기 어렵게 학습하고, 판별 모델은 실제 데이터와 생성된 데이터를 더욱 잘 구분하게 학습한다.

하지만 ELECTRA의 생성 모델은 마스킹된 언어 모델을 통해 학습되며, 판별 모델은 각 토큰이 바뀐 토큰인지 아니면 원본 토큰인지를 구분하도록 학습한다.

마지막으로 GAN은 완전한 노이즈 벡터를 입력받아 생성하지만, ELECTRA의 생성 모델은 일부 토큰이 마스크 처리된 텍스트를 입력으로 받는다.

생성 모델과 판별 모델은 모두 트랜스포머 인코더 구조를 따른다. 따라서 두 모델이 같은 개수의 계층으로 구성돼 있다면 가중치를 공유할 수 있어 더 빠르게 학습할 수 있다. 그러나 두 모델이 가중치를 완전히 공유하면 생성 모델의 성능이 너무 높아져서 판별 모델이 학습할 수 없게 된다. 그러므로 ELECTRA는 생성 모델을 판별 모델의 크기를 1/2에서 1/4 크기로 바꿔 설정하고 모든 가중치를 공유하는 대신 임베딩 계층의 가중치만 공유한다.

ELECTRA는 사전 학습이 완료되면 생성 모델을 사용하지 않고 오직 판별 모델만을 사용해 다운스트림 작업을 수행한다. 판별 모델은 트랜스포머 인코더로 구성되어 있으며, BERT와 동일한 구조를 갖는다. 다운스트림 작업은 BERT와 동일한 방식으로 미세 조정한다.

ELECTRA GLUE 평가

ELECTRA는 GLUE 평가에서 높은 점수를 기록한다. electra-small과 bert-small은 동일한 구조와 가중치 개수를 가지지만, electra-small은 bert-small보다 5% 높은 점수를 기록했다.

비슷한 방식으로, electra-base와 bert-base는 약 3% 차이, electra-large와 bert-large는 약 2% 차이로 ELECTRA 모델이 더 좋은 점수를 기록했다.

또한 12시간 학습한 electra-small 모델은 bert-small 모델이 4일 학습 결과보다 더 좋은 GLUE 점수를 기록했다. 이는 ELECTRA가 더 효율적으로 학습되어 더 높은 성능을 보인다는 것을 의미한다.

T5

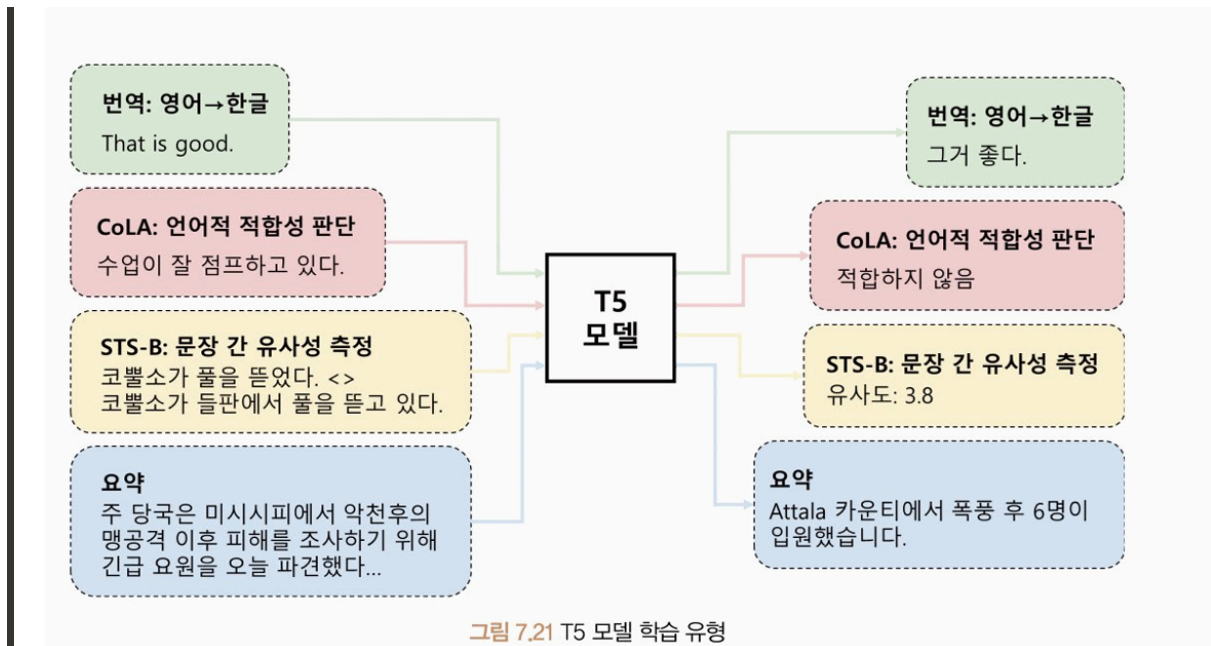
T5(Text-to-Text Transfer Transformer)는 2019년 구글에서 발표한 자연어 처리 분야의 딥러닝 모델로 트랜스포머 구조를 기반으로 한다.

T5는 인코더-디코더 모델 구조를 바탕으로 GLUE, SuperGLUE, CNN/DM(Cable News Network/Daily Mail) 등의 데이터세트에서 **SOTA(State-of-the-art)**를 달성했으며 다양한 자연어 처리 작업에서 높은 성능을 보이는 모델이다.

기존의 자연어 처리 모델은 대부분 입력 문장을 벡터나 행렬로 변환한 뒤, 이를 이용해 출력 문장을 생성하는 방식이거나 출력값이 클래스 또는 입력값의 일부를 반환하는 형식으로 동작했다.

이와 달리 T5는 입력과 출력을 모두 토큰 시퀀스로 처리하는 **텍스트-텍스트(Text-to-Text)** 구조다. 따라서 입력과 출력의 형태를 자유롭게 다룰 수 있으며, 모델 구조상 유연성과 확장성이 뛰어나기 때문에 새로운 자연어 처리 작업에서도 쉽게 적용할 수 있다.

이러한 특징 덕분에 T5는 자연어 처리 분야에서 다양한 작업에 사용할 수 있다. 대표적인 학습 작업으로는 문장 번역, 요약, 질의응답, 텍스트 분류 등이 있다.



T5는 입력과 출력이 모두 토큰(텍스트) 시퀀스이기 때문에 입력과 출력 간의 관계를 더욱 세밀하게 다룰 수 있다. T5는 사전 학습 후 미세 조정 단계에서 해당 작업의 데이터를 이용해 모델을 조정해 최적의 성능을 얻을 수 있다.

T5는 입력과 출력을 모두 텍스트 시퀀스로 처리하는 텍스트-텍스트 모델이므로 학습을 위한 데이터세트는 원본 문장과 대상 문장을 사용한다. T5는 **C4(Colossal Clean Crawled Corpus)** 데이터세트를 활용해 다양한 자연어 처리 작업을 수행할 수 있게 사전 학습됐다.

사전 학습 방식은 비지도 학습 방식으로 입력 문장의 일부 구간을 마스킹해 입력 시퀀스를 처리하며, 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성된다.

이때 문장마다 유일한 마스크 토큰을 의미하는 **센티널 토큰(Sentinel Token)**이 사용된다. 센티널 토큰은 `<extra_id_0>`, `<extra_id_1>` 과 같이 0부터 99까지 100개의 기본값을 사용한다.

T5는 이러한 마스킹 토큰을 예측하는 것을 목적으로 사전 학습되며, 사전 학습이 완료된 후 미세 조정을 통해 다양한 자연어 처리 작업을 수행한다.

T5의 미세 조정은 지도 학습 방식으로 학습된다. 번역, 언어 수용성, 문장 유사도, 문서 요약 등 다양한 작업에 활용할 수 있다. T5 모델의 입력과 출력은 텍스트 시퀀스 토큰들로 이뤄져 있으며, 인코더-디코더 T5 모델에 입력된다.

이 과정에서 문장이 인코딩되기 전에 'translate English to German:' 또는 'summerize:'와 같은 작업 토큰을 문장 앞에 추가한다. 이러한 방식은 작업 토큰도 함께 학습해 다양한 자연어 처리 작업에서 높은 성능을 발휘할 수 있게 된다.

T5 빔 서치 디코딩

빔 개수(num_beams)는 **빔 서치(Beam Search)** 알고리즘의 빔 크기를 의미한다. 빔 서치 알고리즘은 디코더 모델이 생성한 다수의 후보 단어 시퀀스 중에서 가장 높은 확률을 가진 시퀀스를 선택해 출력한다.

디코더 모델은 다음 단어를 예측하는 과정에서 다수의 후보 단어를 생성한다. 이때 빔서치 알고리즘은 미리 지정한 빔 크기만큼의 후보 단어 시퀀스만을 유지하고, 나머지 후보 시퀀스들은 삭제한다. 이후 다음 단어를 예측하면서 빔 크기에 맞게 후보 시퀀스들을 업데이트하며, 최종적으로 가장 높은 확률을 가진 시퀀스를 선택한다.

반복 페널티(repetition_penalty)는 중복 토큰 생성을 제어하는 값이다. 이 값이 높을수록 중복 토큰 생성이 억제된다.

길이 페널티(length_penalty)는 생성된 시퀀스 길이에 대한 보상을 제어한다. 이 값이 높을수록 더욱 긴 시퀀스가 생성된다.

조기 중단(early_stopping)은 최대 길이에 도달하기 전 EOS 토큰이 생성되는 경우 중단한다.